DATA PIPELINE

데이터 수집·활용 절차

01 출처

- 드론·CCTV 영상 (가상 드론 RTSP / 더미 MP4)
- 행정구역 GeoJSON
- 사전학습: VisDrone
- 실장비 없음·더미 dev만

02 수집

- Flask 분석 서버
- YOLOv8m 탐지
- PointTracker 추적·체류
- 라이브: ffmpeg 녹화
- 박스 좌표 파일 생성

03 전송

JSON: 인원·in/out·체류

- 내부 토큰 인증 · 실패 시 5분 재전송
- 1초마다 전송

04 가공

- Spring 관리자 앱
- 유효성 확인 · 구역별 누적
- 1분 → 시간 → 일 집계

05 저장

- MySQL: 통계·이벤트
- 메모리: 현재 인원
- 파일: MP4 · JSON.gz

06 활용

- 지도·그래프·대시보드·엑셀 (두 앱)
- 이벤트 알림 (관리자)
- 다시보기 (관리자 전용)

저장·전달 경로 3가지 · 관리자 앱은 분석 서버에 캠 시작·정지를 지시(제어 흐름)

통계

1초 결과 → 1분 통계 → 시간별 → 일별 → MySQL 저장 (지도·그래프·엑셀)

현재 인원

최신 초 인원 → admin 메모리 → 5초마다 SSE → 화면 (15초 지나면 1분 평균)

영상·좌표

MP4 + JSON.gz → 공유 폴더(NFS) → 권한 확인 후 nginx 전달 → 다시보기

영상은 1초 결과 → 1분·시간·일 통계로 모여 지도·그래프·엑셀·이벤트 알림으로 나가고, 영상 원본과 좌표는 다시보기로 쓰인다

PROCESSING·STORAGE·UTILIZATION

데이터 수집·활용 절차(가공·저장)

가공 · 저장 (관리자 앱 Spring admin)

수신·누적
구역·분별로 합산

1분 통계
분 끝+5초 마감

시간별
매시 01분 30초 집계

일별
매일 00:05 집계

수신·계산 규칙

- 삭제·정지·없는 캠 결과 제외
- 마감된 분의 늦은 결과 제외
- 밀집도 = 평균 인원 ÷ 면적(명/㎡)
- 끊긴 구간은 0명 아닌 '빈 구간'

이벤트 판정 (1분마다)

- 혼잡: 주의 0.7 · 위험 1.5 명/㎡
- 급증: 10분 전보다 +50% 이상
- 캠 이상: 오류 또는 결과 없음
- 예) 100㎡·80명 → 0.8명/㎡(설명용)

저장소와 보관 정책 (명세 기준 값)

메모리 (admin)
분별 누적기 · 현재 인원(15초 이내, 5초 갱신)

MySQL
통계·이벤트·기록 · 1분 통계 90일 · 녹화 30일

공유 폴더 (NFS)
MP4 · 박스 좌표 JSON.gz · 라이브 32일째 정리

분석 서버 로컬
더미 분석 캐시 · 녹화 중 임시 파일

활용 (사용자 · 관리자 · 최고 관리자)

사용자
통계만 · 지도·그래프·엑셀 (영상 열람 불가)

관리자
통계 + 영상 + 이벤트 · 캠 가동·정지, 구역·캠 관리

최고 관리자
관리자 전체 + 계정 관리 · 작업 기록 조회

저장된 데이터를 내보내는 4가지 방법

지도·그래프·대시보드
점을 눌러 연→월→일→시간→분 · SSE 5초 갱신

엑셀 다운로드
POI SXSSF 스트리밍 · 100만 행 초과 시 거절

이벤트 알림·조치
미확인 → 확인 → 조치완료 · 그 시점 다시보기

다시보기 (영상 + 박스)
Spring은 권한만 확인, nginx가 파일 전달

최종 목적(설계 목표): 누적 데이터로 시간대별·계절별 구역 활성화 정도를 보고 상권 분석·도시 설계에 활용한다